

HRMS

Human Resource Management System

Technical Design Document

Next.js 15 · Express.js · Prisma 7 · PostgreSQL · Bun Monorepo

Version	v1.0
Date	July 2026
Status	Draft — For Review
Classification	Confidential

Catalog

1. Purpose & Scope	3
2. Technology Stack Overview	4
3. Monorepo Structure	5
3.1 Root Workspace Configuration	5
3.2 Full Folder Structure	5
4. Frontend Architecture	8
4.1 Next.js 15 — App Router Conventions	8
4.2 Axios Instance Configuration	8
4.3 TanStack Query — Setup & Conventions	8
4.4 shadcn/ui & Tailwind Conventions	9
4.5 Toast Notifications (Sonner)	10
5. Backend Architecture	11
5.1 Express.js App Setup	11
5.2 Route → Controller → Service Pattern	11
5.3 Zod Validation Middleware	12
5.4 Global Error Handler	13
5.5 Standard API Response Shape	13
6. Prisma 7 & Database Design	15
6.1 Prisma Configuration (prisma.config.ts)	15
6.2 Prisma Client Singleton	15
6.3 Prisma Schema Conventions	15
6.4 Config Service — Metadata Read Layer	16
7. Authentication & Authorization	18
7.1 Auth Strategy Overview	18
7.2 Login Flow	18
7.3 Token Refresh Flow	19
7.4 authenticate Middleware	19
7.5 authorize Middleware — RBAC	20
7.6 RBAC Role Matrix	20
8. API Design Conventions	22
8.1 URL Structure	22
8.2 Pagination Convention	22
8.3 Shared Zod Schemas (packages/validators)	23
9. Environment Configuration	24
9.1 Backend Environment Variables	24
9.2 Frontend Environment Variables	24
9.3 Environment Validation (Zod)	24
10. Security Design	25
11. Coding Standards & Conventions	26
11.1 TypeScript	26
11.2 Naming Conventions	26
11.3 Backend Rules	26
11.4 Frontend Rules	26
12. Local Development Setup	28
13. Document Control	29

1. Purpose & Scope

This Technical Design Document (TDD) defines the technology stack, architecture, folder structure, authentication flow, API conventions, database approach, coding standards, and infrastructure for the HRMS custom ERP application.

It is the primary reference for frontend and backend engineers during implementation, and should be read alongside the Solution Design Document (SDD v2.0) and the Functional Design Document (FDD v1.0).

2. Technology Stack Overview

Layer	Technology	Version	Purpose
Frontend Framework	Next.js (App Router)	15	React framework with SSR, RSC, and file-based routing
Frontend Language	TypeScript	5.x	Type safety across the entire frontend codebase
Styling	Tailwind CSS	4.x	Utility-first CSS — no custom stylesheet files
Component Library	shadcn/ui	Latest	Pre-built accessible components built on Radix UI
Server State	TanStack Query	5.x	Data fetching, caching, background sync, and mutations
HTTP Client	Axios	1.x	API calls with withCredentials:true for cookie-based auth
Notifications	Sonner	Latest	Toast notification system
Icons	Lucide React	Latest	Icon library
Backend Framework	Express.js	5.x	Node.js HTTP server and routing
Backend Language	TypeScript	5.x	Type safety across the entire backend codebase
ORM	Prisma	7.x	Type-safe database access with prisma.config.ts + PrismaPg adapter
Database	PostgreSQL	16+	Primary relational database
Validation	Zod	3.x	Runtime schema validation for all API request bodies
Authentication	JWT	—	httpOnly cookie-based access + refresh token strategy
Password Hashing	bcryptjs	Latest	Secure password hashing with configurable salt rounds
Runtime & Monorepo	Bun	Latest	Monorepo workspace manager, runtime, and package manager

3. Monorepo Structure

The project is managed as a Bun workspace monorepo. The root package.json declares the workspaces. All packages share the same node_modules hoisting via Bun.

3.1 Root Workspace Configuration

```
// package.json (root)
{
  "name": "hrms",
  "private": true,
  "workspaces": ["frontend", "backend", "packages/*"],
  "scripts": {
    "dev": "bun run --filter=* dev",
    "dev:frontend": "bun run --filter=frontend dev",
    "dev:backend": "bun run --filter=backend dev",
    "build": "bun run --filter=* build",
    "lint": "bun run --filter=* lint",
    "typecheck": "bun run --filter=* typecheck"
  }
}
```

3.2 Full Folder Structure

```
hrms/
├── package.json           # Bun workspace root
├── bun.lockb
├── .env.example
├── docker-compose.yml     # Local PostgreSQL
├── packages/
│   ├── types/             # Shared TypeScript types & interfaces
│   │   ├── src/
│   │   │   ├── employee.ts
│   │   │   ├── bpv.ts
│   │   │   ├── task.ts
│   │   │   ├── asset.ts
│   │   │   ├── reward.ts
│   │   │   └── index.ts
│   │   └── package.json
│   └── validators/        # Shared Zod schemas (used by both frontend &
backend)
│   ├── src/
│   │   ├── employee.schema.ts
│   │   ├── bpv.schema.ts
│   │   ├── task.schema.ts
│   │   ├── asset.schema.ts
│   │   └── index.ts
│   └── package.json
└── utils/                 # Shared pure utilities
    └── src/
```

```
├── date.ts
├── format.ts
├── index.ts
├── package.json
├── frontend/                                # Next.js 15 App
│   ├── package.json
│   ├── next.config.ts
│   ├── tailwind.config.ts
│   ├── tsconfig.json
│   └── src/
│       ├── app/                            # App Router
│       │   ├── (auth)/
│       │   │   ├── login/page.tsx
│       │   │   └── layout.tsx
│       │   ├── (dashboard)/
│       │   │   ├── layout.tsx
│       │   │   ├── page.tsx                # Dashboard home
│       │   │   ├── employee/
│       │   │   ├── bpv/
│       │   │   ├── tasks/
│       │   │   ├── time/
│       │   │   ├── assets/
│       │   │   ├── salary/
│       │   │   ├── performance/
│       │   │   ├── rewards/
│       │   │   ├── beneficiaries/
│       │   │   ├── reports/
│       │   │   ├── admin/
│       │   │   │   ├── config/
│       │   │   │   └── audit/
│       │   ├── api/                        # Next.js Route Handlers (thin proxy only)
│       │   ├── layout.tsx
│       │   └── globals.css
│       ├── components/
│       │   ├── ui/                        # shadcn/ui generated components
│       │   ├── layout/                    # Sidebar, Navbar, PageHeader
│       │   ├── forms/                     # Reusable form components
│       │   ├── tables/                    # DataTable with TanStack Table
│       │   ├── charts/                    # Analytics chart components
│       │   ├── shared/                    # Badges, StatusPills, ConfirmDialog
│       │   └── hooks/                     # TanStack Query hooks per module
│       ├── useEmployee.ts
│       ├── useBPV.ts
│       ├── useTasks.ts
│       ├── useAssets.ts
│       ├── useRewards.ts
│       └── useConfig.ts
│       ├── lib/
│       │   ├── api.ts                     # Axios instance + interceptors
│       │   ├── queryClient.ts             # TanStack Query client config
│       │   ├── utils.ts                   # cn() and other helpers
│       ├── providers/
│       │   └── QueryProvider.tsx
│       └── types/                          # Frontend-only type extensions
├── backend/                                # Express.js API
│   ├── package.json
│   ├── tsconfig.json
│   ├── prisma.config.ts                   # Prisma 7 config with PrismaPg adapter
│   └── prisma/
```

```
├── schema.prisma
├── migrations/
├── src/
│   ├── index.ts           # Server entry point
│   ├── app.ts             # Express app setup & middleware
│   ├── routes/            # Route registration
│   │   ├── index.ts
│   │   ├── auth.routes.ts
│   │   ├── employee.routes.ts
│   │   ├── bpv.routes.ts
│   │   ├── task.routes.ts
│   │   ├── time.routes.ts
│   │   ├── salary.routes.ts
│   │   ├── asset.routes.ts
│   │   ├── reward.routes.ts
│   │   ├── report.routes.ts
│   │   └── admin.routes.ts
│   ├── controllers/       # Request handlers (thin – delegate to
services)
│   │   ├── services/      # All business logic lives here
│   │   │   ├── bpv.service.ts
│   │   │   ├── salary.service.ts
│   │   │   ├── efficiency.service.ts
│   │   │   ├── config.service.ts # Metadata config read layer
│   │   │   └── ...
│   │   ├── middleware/
│   │   │   ├── authenticate.ts # JWT cookie validation
│   │   │   ├── authorize.ts    # RBAC permission check
│   │   │   ├── validate.ts     # Zod schema validation
│   │   │   └── errorHandler.ts # Global error handler
│   │   ├── lib/
│   │   │   ├── prisma.ts      # Prisma client singleton
│   │   │   ├── jwt.ts         # Token sign / verify helpers
│   │   │   └── logger.ts
│   │   ├── config/
│   │   │   └── env.ts         # Validated env vars via Zod
```

4. Frontend Architecture

4.1 Next.js 15 — App Router Conventions

The frontend uses the Next.js 15 App Router exclusively. No Pages Router. All routes are defined by folder structure under `src/app/`.

- Route groups: (auth) for unauthenticated pages, (dashboard) for all authenticated module pages.
- Layout files (layout.tsx) provide persistent shell UI — sidebar, navbar — at the dashboard group level.
- Server Components (RSC) are used for layout shells and initial page frames. Client Components ('use client') are used for all interactive UI.
- Loading states use loading.tsx files per route segment for automatic Suspense boundaries.
- Error boundaries use error.tsx files per route segment.
- Next.js Route Handlers (`src/app/api/`) are used only as a thin proxy if needed — all business logic is in the Express backend.

4.2 Axios Instance Configuration

A single Axios instance is created and exported from `src/lib/api.ts`. All API calls in the application use this instance — never raw fetch or a new axios() call.

```
// src/lib/api.ts
import axios from 'axios'

const api = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL, // e.g. http://localhost:4000/api/v1
  withCredentials: true,                  // send httpOnly cookies
  automatically
  headers: { 'Content-Type': 'application/json' },
})

// Response interceptor — auto-refresh on 401
api.interceptors.response.use(
  (res) => res,
  async (error) => {
    const original = error.config
    if (error.response?.status === 401 && !original._retry) {
      original._retry = true
      await api.post('/auth/refresh') // refresh token is sent via httpOnly
      return api(original)           // retry original request
    }
    return Promise.reject(error)
  }
)

export default api
```

4.3 TanStack Query — Setup & Conventions

TanStack Query manages all server state. The QueryClient is configured once and provided at the root via QueryProvider.

```
// src/lib/queryClient.ts
import { QueryClient } from '@tanstack/react-query'

export const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 60 * 1000, // 1 minute
      retry: 1,
      refetchOnWindowFocus: false,
    },
  },
})
```

Query key conventions — every module uses a structured key factory:

```
// src/hooks/useEmployee.ts — query key factory pattern
export const employeeKeys = {
  all: () => ['employees'] as const,
  list: (filters: object) => ['employees', 'list', filters] as const,
  detail: (id: string) => ['employees', 'detail', id] as const,
  bpv: (id: string) => ['employees', 'bpv', id] as const,
}

export function useEmployee(id: string) {
  return useQuery({
    queryKey: employeeKeys.detail(id),
    queryFn: () => api.get(`/employees/${id}`).then(r => r.data.data),
  })
}

export function useUpdateEmployee(id: string) {
  const qc = useQueryClient()
  return useMutation({
    mutationFn: (data) => api.patch(`/employees/${id}`, data),
    onSuccess: () => {
      qc.invalidateQueries({ queryKey: employeeKeys.detail(id) })
      toast.success('Profile updated')
    },
    onError: (err) => toast.error(err.response?.data?.error?.message ?? 'Update failed'),
  })
}
```

4.4 shadcn/ui & Tailwind Conventions

- All UI components are built from shadcn/ui primitives installed into src/components/ui/.
- Custom components extend shadcn/ui — never re-implement base components from scratch.
- Tailwind CSS only — no inline styles, no CSS modules, no styled-components.
- The cn() utility (clsx + tailwind-merge) is used for all conditional class merging.
- Dark mode support via Tailwind's dark: variant — theme toggled via a class on <html>.

- All form inputs use React Hook Form + shadcn/ui Form components + Zod resolver.

4.5 Toast Notifications (Sonner)

All user-facing feedback uses Sonner toast notifications. The Toaster component is mounted once at the root layout. Toasts are called from TanStack Query mutation callbacks.

```
// Root layout
import { Toaster } from 'sonner'

export default function RootLayout({ children }) {
  return (
    <html>
      <body>
        {children}
        <Toaster richColors position='top-right' />
      </body>
    </html>
  )
}

// Usage in mutation callbacks
import { toast } from 'sonner'
toast.success('Employee created successfully')
toast.error('Failed to save changes')
toast.loading('Calculating BPV score...')
```

5. Backend Architecture

5.1 Express.js App Setup

The Express app is configured in `src/app.ts` and started in `src/index.ts`. All middleware is registered before routes.

```
// src/app.ts
import express from 'express'
import cookieParser from 'cookie-parser'
import cors from 'cors'
import { errorHandler } from '../middleware/errorHandler'
import routes from './routes'

const app = express()

app.use(cors({
  origin: process.env.FRONTEND_URL,
  credentials: true, // allow cookies cross-origin
}))
app.use(express.json())
app.use(cookieParser())

app.use('/api/v1', routes)
app.use(errorHandler) // must be last

export default app

// src/index.ts
import app from './app'
const PORT = process.env.PORT ?? 4000
app.listen(PORT, () => console.log(`HRMS API running on port ${PORT}`))
```

5.2 Route → Controller → Service Pattern

Every module follows the same three-layer pattern. Routes declare endpoints and apply middleware. Controllers handle request/response. Services contain all business logic and database access.

```
// routes/employee.routes.ts
import { Router } from 'express'
import { authenticate } from '../middleware/authenticate'
import { authorize } from '../middleware/authorize'
import { validate } from '../middleware/validate'
import { createEmployeeSchema, updateEmployeeSchema } from '@hrms/validators'
import * as ctrl from '../controllers/employee.controller'

const router = Router()

router.get('/', authenticate, authorize('HR_ADMIN', 'MANAGER'), ctrl.list)
router.post('/', authenticate, authorize('HR_ADMIN'),
  validate(createEmployeeSchema), ctrl.create)
router.get('/:id', authenticate, authorize('HR_ADMIN', 'MANAGER', 'EMPLOYEE'),
  ctrl.getById)
router.patch('/:id', authenticate, authorize('HR_ADMIN', 'EMPLOYEE'),
```

```

validate(updateEmployeeSchema), ctrl.update)

export default router

// controllers/employee.controller.ts
import { Request, Response, NextFunction } from 'express'
import * as employeeService from '../services/employee.service'

export async function create(req: Request, res: Response, next: NextFunction) {
  try {
    const employee = await employeeService.create(req.body)
    res.status(201).json({ success: true, data: employee })
  } catch (err) {
    next(err) // pass to global error handler
  }
}

// services/employee.service.ts
import { prisma } from '../lib/prisma'

export async function create(data: CreateEmployeeInput) {
  // validation already done by Zod middleware
  const employee = await prisma.employee.create({ data })
  // trigger BPV calculation
  await bpvService.calculate(employee.id)
  return employee
}

```

5.3 Zod Validation Middleware

All incoming request bodies are validated by a Zod schema before reaching the controller. Invalid requests are rejected with a 422 response before any business logic runs.

```

// middleware/validate.ts
import { ZodSchema } from 'zod'
import { Request, Response, NextFunction } from 'express'

export function validate(schema: ZodSchema) {
  return (req: Request, res: Response, next: NextFunction) => {
    const result = schema.safeParse(req.body)
    if (!result.success) {
      return res.status(422).json({
        success: false,
        error: {
          code: 'VALIDATION_ERROR',
          message: 'Invalid request data',
          details: result.error.flatten().fieldErrors,
        },
      })
    }
    req.body = result.data // replace with parsed + coerced data
    next()
  }
}

```

5.4 Global Error Handler

All errors are passed to the global error handler via `next(err)`. Services throw typed `AppError` instances. Prisma errors and unexpected errors are also caught and normalized.

```
// middleware/errorHandler.ts
import { Request, Response, NextFunction } from 'express'
import { Prisma } from '@prisma/client'

export class AppError extends Error {
  constructor(
    public statusCode: number,
    public code: string,
    message: string,
  ) { super(message) }
}

export function errorHandler(err: unknown, req: Request, res: Response, _next: NextFunction) {
  if (err instanceof AppError) {
    return res.status(err.statusCode).json({
      success: false,
      error: { code: err.code, message: err.message }
    })
  }
  if (err instanceof Prisma.PrismaClientKnownRequestError) {
    if (err.code === 'P2002') { // unique constraint
      return res.status(409).json({
        success: false,
        error: { code: 'DUPLICATE_ENTRY', message: 'A record with this value already exists' }
      })
    }
  }
  console.error(err)
  res.status(500).json({
    success: false,
    error: { code: 'INTERNAL_ERROR', message: 'An unexpected error occurred' }
  })
}
```

5.5 Standard API Response Shape

Every API response — success or error — follows the same envelope structure. This is enforced by convention in all controllers.

```
// Success
{
  "success": true,
  "data": { ... } // single object
}

// Success — paginated list
{
  "success": true,
  "data": {
```

```
    "items": [...],
    "total": 150,
    "page": 1,
    "limit": 20
  }
}

// Error
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Human-readable message",
    "details": { "field": ["error message"] } // optional, for validation errors
  }
}
```

6. Prisma 7 & Database Design

6.1 Prisma Configuration (prisma.config.ts)

Prisma 7 uses a `prisma.config.ts` file with the `PrismaPg` adapter for direct PostgreSQL access via the `pg` driver.

```
// prisma.config.ts (backend root)
import { defineConfig } from 'prisma/config'
import { PrismaPg }      from '@prisma/adapter-pg'
import { Pool }          from 'pg'

const pool = new Pool({ connectionString: process.env.DATABASE_URL })

export default defineConfig({
  earlyAccess: true,
  schema: './prisma/schema.prisma',
  adapter: () => new PrismaPg(pool),
})
```

6.2 Prisma Client Singleton

```
// src/lib/prisma.ts
import { PrismaClient } from '@prisma/client'
import { PrismaPg }      from '@prisma/adapter-pg'
import { Pool }          from 'pg'

const pool = new Pool({ connectionString: process.env.DATABASE_URL })
const adapter = new PrismaPg(pool)

const globalForPrisma = globalThis as unknown as { prisma?: PrismaClient }

export const prisma = globalForPrisma.prisma ?? new PrismaClient({ adapter })

if (process.env.NODE_ENV !== 'production') {
  globalForPrisma.prisma = prisma
}
```

6.3 Prisma Schema Conventions

Key conventions applied across all models in `schema.prisma`:

- All primary keys use `cuid()` — collision-resistant, URL-safe, sortable strings.
- All tables include `createdAt DateTime @default(now())` and `updatedAt DateTime @updatedAt`.
- Soft deletes where applicable — `deletedAt DateTime?` field. Records with `deletedAt` set are excluded from default queries via a global query extension.
- All configuration/master tables include `effectiveFrom`, `effectiveTo`, `isActive`, `createdBy`, and `changeReason` fields to support the metadata versioning model.

- Enums are defined in the schema for fixed value sets (EmployeeStatus, AssetStatus, TaskStatus, etc.).

Sample model showing conventions:

```
// prisma/schema.prisma (excerpt)

model Employee {
  id            String           @id @default(cuid())
  employeeCode  String           @unique
  fullName      String
  email         String           @unique
  status        EmployeeStatus  @default(ACTIVE)
  departmentId  String
  department    Department       @relation(fields: [departmentId], references:
[id])
  bpvScores     BpvScore[]
  skills        EmployeeSkill[]
  qualifications EmployeeQualification[]
  createdAt     DateTime         @default(now())
  updatedAt     DateTime         @updatedAt
  deletedAt     DateTime?
  @@index([departmentId])
  @@index([status])
}

// Configuration table following versioning pattern
model BpvWeightConfig {
  id            String           @id @default(cuid())
  educationPct  Float
  experiencePct Float
  orgProfilePct Float
  skillsPct     Float
  certsPct      Float
  effectiveFrom DateTime
  effectiveTo   DateTime?
  isActive      Boolean @default(true)
  changeReason  String
  createdBy     String
  createdAt     DateTime @default(now())
}

enum EmployeeStatus {
  ACTIVE
  ON_NOTICE
  EXITED
}
```

6.4 Config Service — Metadata Read Layer

All business services read configuration entities through a dedicated ConfigService. This service handles version resolution (fetching the config active at a given date) and in-memory caching.

```
// services/config.service.ts
import { prisma } from '../lib/prisma'
```



```
// In-memory config cache — invalidated on admin update
const cache = new Map<string, { data: unknown; cachedAt: number }>()
const CACHE_TTL = 5 * 60 * 1000 // 5 minutes

export async function getActiveConfig<T>({
  model: string,
  asOf: Date = new Date()
}): Promise<T> {
  const cacheKey = `${model}:${asOf.toISOString().split('T')[0]}`
  const cached = cache.get(cacheKey)
  if (cached && Date.now() - cached.cachedAt < CACHE_TTL) {
    return cached.data as T
  }
  const record = await (prisma as any)[model].findFirst({
    where: {
      isActive: true,
      effectiveFrom: { lte: asOf },
      OR: [{ effectiveTo: null }, { effectiveTo: { gt: asOf } }],
    },
    orderBy: { effectiveFrom: 'desc' },
  })
  if (!record) throw new AppError(500, 'CONFIG_MISSING', `No active config for ${model}`)
  cache.set(cacheKey, { data: record, cachedAt: Date.now() })
  return record as T
}

export function invalidateConfigCache(model: string) {
  for (const key of cache.keys()) {
    if (key.startsWith(model)) cache.delete(key)
  }
}
```

7. Authentication & Authorization

7.1 Auth Strategy Overview

Aspect	Implementation
Token Type	JWT — two tokens: Access Token + Refresh Token
Access Token TTL	15 minutes — configurable via DB (session config table)
Refresh Token TTL	7 days — configurable via DB
Storage	httpOnly cookies — never localStorage, never response body
Cookie Flags	httpOnly: true, secure: true (prod), sameSite: 'strict'
Cross-Origin	CORS credentials: true, Axios withCredentials: true
Password Hashing	bcryptjs with salt rounds from DB config (default: 12)
RBAC	Role checked on every protected route via authorize() middleware

7.2 Login Flow

```
// POST /api/v1/auth/login
// services/auth.service.ts

export async function login(email: string, password: string, res: Response) {
  const user = await prisma.user.findUnique({ where: { email } })
  if (!user || !await bcrypt.compare(password, user.passwordHash)) {
    throw new AppError(401, 'INVALID_CREDENTIALS', 'Invalid email or password')
  }

  const sessionConfig = await configService.getActiveConfig('SessionConfig')

  const accessToken = jwt.sign(
    { sub: user.id, role: user.role, emp: user.employeeId },
    process.env.JWT_ACCESS_SECRET!,
    { expiresIn: sessionConfig.accessTokenTtlMinutes * 60 }
  )

  const refreshToken = jwt.sign(
    { sub: user.id },
    process.env.JWT_REFRESH_SECRET!,
    { expiresIn: sessionConfig.refreshTokenTtlDays * 86400 }
  )

  // Store refresh token hash in DB for rotation validation
  await prisma.refreshToken.create({
    data: { userId: user.id, tokenHash: await bcrypt.hash(refreshToken, 10),
    expiresAt: ... }
  })

  const cookieOpts = {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'strict' as const,
  }

  res.cookie('access_token', accessToken, { ...cookieOpts, maxAge:
```

```
sessionConfig.accessTokenTtlMinutes * 60000 })
  res.cookie('refresh_token', refreshToken, { ...cookieOpts, maxAge:
sessionConfig.refreshTokenTtlDays * 86400000 })

  return { id: user.id, role: user.role, name: user.fullName }
}
```

7.3 Token Refresh Flow

```
// POST /api/v1/auth/refresh
export async function refresh(refreshToken: string, res: Response) {
  let payload: { sub: string }
  try {
    payload = jwt.verify(refreshToken, process.env.JWT_REFRESH_SECRET!) as any
  } catch {
    throw new AppError(401, 'INVALID_REFRESH_TOKEN', 'Session expired - please
log in again')
  }

  // Validate against stored token hash (rotation - old token invalidated after
use)
  const stored = await prisma.refreshToken.findFirst({
    where: { userId: payload.sub, revokedAt: null, expiresAt: { gt: new
Date() } }
  })
  const valid = stored && await bcrypt.compare(refreshToken, stored.tokenHash)
  if (!valid) throw new AppError(401, 'INVALID_REFRESH_TOKEN', 'Invalid session')

  // Revoke old token
  await prisma.refreshToken.update({ where: { id: stored.id }, data: { revokedAt:
new Date() } })

  // Issue new access + refresh tokens (rotation)
  // ... (same as login flow above)
}
```

7.4 authenticate Middleware

```
// middleware/authenticate.ts
import jwt from 'jsonwebtoken'
import { Request, Response, NextFunction } from 'express'

export interface AuthUser {
  id: string
  role: string
  emp: string // employeeId
}

declare global {
  namespace Express {
    interface Request { user?: AuthUser }
  }
}
```

```

    }
  }

  export function authenticate(req: Request, res: Response, next: NextFunction) {
    const token = req.cookies['access_token']
    if (!token) return res.status(401).json({ success:false,
error:{ code:'UNAUTHENTICATED', message:'Not authenticated' } })

    try {
      req.user = jwt.verify(token, process.env.JWT_ACCESS_SECRET!) as AuthUser
      next()
    } catch {
      res.status(401).json({ success:false, error:{ code:'TOKEN_EXPIRED',
message:'Session expired' } })
    }
  }
}

```

7.5 authorize Middleware — RBAC

```

// middleware/authorize.ts
import { Request, Response, NextFunction } from 'express'

export function authorize(...roles: string[]) {
  return (req: Request, res: Response, next: NextFunction) => {
    if (!req.user) {
      return res.status(401).json({ success:false,
error:{ code:'UNAUTHENTICATED', message:'Not authenticated' } })
    }
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ success:false, error:{ code:'FORBIDDEN',
message:'You do not have permission to perform this action' } })
    }
    // For EMPLOYEE role: further scope check (can only access own data)
    if (req.user.role === 'EMPLOYEE') {
      const targetId = req.params.id ?? req.body.employeeId
      if (targetId && targetId !== req.user.emp) {
        return res.status(403).json({ success:false, error:{ code:'FORBIDDEN',
message:'Access denied' } })
      }
    }
    next()
  }
}

```

7.6 RBAC Role Matrix

Role	API Access Scope
EMPLOYEE	Own profile data only. Own tasks, time entries, assets, beneficiaries, performance, rewards.
MANAGER	Own data + team data (employees reporting to them). Task assignment and review. Team performance.

Role	API Access Scope
HR_ADMIN	All employee data. All configuration entities (HR domain). All assets and compliance data.
FINANCE_ADMIN	Salary data. Reward calculations. Beneficiary payout routing. Finance configuration entities.
SYSTEM_ADMIN	All endpoints. User management. Audit logs. System configuration.

8. API Design Conventions

8.1 URL Structure

All endpoints are prefixed with `/api/v1/`. RESTful resource naming. Nested resources for ownership relationships.

Method	Endpoint	Description
GET	<code>/api/v1/employees</code>	List employees (paginated, filterable)
POST	<code>/api/v1/employees</code>	Create employee
GET	<code>/api/v1/employees/:id</code>	Get employee by ID
PATCH	<code>/api/v1/employees/:id</code>	Update employee (partial update)
GET	<code>/api/v1/employees/:id/bpv</code>	Get BPV score and history
POST	<code>/api/v1/employees/:id/bpv/recalculate</code>	Trigger manual BPV recalculation
GET	<code>/api/v1/tasks</code>	List tasks (filtered by role)
POST	<code>/api/v1/tasks</code>	Create task (manager)
PATCH	<code>/api/v1/tasks/:id/submit</code>	Employee submits task completion
PATCH	<code>/api/v1/tasks/:id/review</code>	Manager reviews task
POST	<code>/api/v1/time-entries</code>	Create time entry
PATCH	<code>/api/v1/time-entries/:id/review</code>	Manager approves/rejects time entry
GET	<code>/api/v1/assets</code>	List assets
POST	<code>/api/v1/assets</code>	Register new asset
PATCH	<code>/api/v1/assets/:id/version</code>	Submit new asset version
GET	<code>/api/v1/admin/config/:entity</code>	Get config entity (admin)
PUT	<code>/api/v1/admin/config/:entity</code>	Update config entity (versioned)
POST	<code>/api/v1/auth/login</code>	Login — issues httpOnly cookies
POST	<code>/api/v1/auth/refresh</code>	Refresh access token
POST	<code>/api/v1/auth/logout</code>	Revoke refresh token, clear cookies

8.2 Pagination Convention

All list endpoints accept query parameters: `page` (default: 1), `limit` (default: 20, max: 100), `sortBy`, `sortOrder` (asc/desc), plus module-specific filter parameters.

```
// Request
GET
/api/v1/employees?page=2&limit=20&departmentId=xxx&status=ACTIVE&sortBy=fullName&sortOrder=asc

// Response
{
  "success": true,
  "data": {
    "items": [...],
    "total": 150,
    "page": 2,
    "limit": 20,
    "pages": 8
  }
}
```

8.3 Shared Zod Schemas (packages/validators)

Zod schemas live in the shared packages/validators package. Both frontend (form validation) and backend (request validation middleware) import from the same package — single source of truth.

```
// packages/validators/src/employee.schema.ts
import { z } from 'zod'

export const createEmployeeSchema = z.object({
  fullName: z.string().min(2).max(100),
  email: z.string().email(),
  dateOfBirth: z.string().date(),
  designationId: z.string().cuid(),
  departmentId: z.string().cuid(),
  managerId: z.string().cuid().optional(),
  dateOfJoining: z.string().date(),
})

export type CreateEmployeeInput = z.infer<typeof createEmployeeSchema>

export const updateEmployeeSchema = createEmployeeSchema.partial()
export type UpdateEmployeeInput = z.infer<typeof updateEmployeeSchema>
```

9. Environment Configuration

9.1 Backend Environment Variables

```
# backend/.env

# Database
DATABASE_URL=postgresql://hrms_user:password@localhost:5432/hrms_db

# JWT
JWT_ACCESS_SECRET=<256-bit-random-string>
JWT_REFRESH_SECRET=<different-256-bit-random-string>

# App
PORT=4000
NODE_ENV=development
FRONTEND_URL=http://localhost:3000

# Bcrypt
BCRYPT_SALT_ROUNDS=12
```

9.2 Frontend Environment Variables

```
# frontend/.env.local
NEXT_PUBLIC_API_URL=http://localhost:4000/api/v1
```

9.3 Environment Validation (Zod)

Backend environment variables are validated at startup using Zod. The app exits immediately if required vars are missing or malformed.

```
// src/config/env.ts
import { z } from 'zod'

const envSchema = z.object({
  DATABASE_URL: z.string().url(),
  JWT_ACCESS_SECRET: z.string().min(32),
  JWT_REFRESH_SECRET: z.string().min(32),
  PORT: z.coerce.number().default(4000),
  NODE_ENV:
    z.enum(['development', 'production', 'test']).default('development'),
  FRONTEND_URL: z.string().url(),
})

export const env = envSchema.parse(process.env)
```


10. Security Design

Threat	Mitigation
XSS (Cross-Site Scripting)	Tokens stored in httpOnly cookies — inaccessible to JavaScript. React escapes output by default.
CSRF (Cross-Site Request Forgery)	sameSite: 'strict' on cookies prevents cross-origin cookie sending. CORS origin whitelist.
Session Hijacking	Short-lived access tokens (15 min). Refresh token rotation — old token invalidated on use.
Brute Force Login	Rate limiting on /auth/login via express-rate-limit (configurable threshold in DB).
SQL Injection	Prisma ORM uses parameterized queries exclusively — raw SQL is prohibited.
Unauthorized Data Access	RBAC on every route + employee scope check — employees can only access own data.
Sensitive Data Exposure	Salary, BPV, and reward data excluded from responses for roles without access. Never logged.
Password Exposure	Passwords hashed with bcryptjs before storage. Password never returned in any API response.
Insecure Dependencies	Bun audit on every CI run. Dependabot alerts enabled on the repository.
Config Tampering	All configuration changes are versioned and logged. No config record can be deleted.

11. Coding Standards & Conventions

11.1 TypeScript

- strict: true in all tsconfig.json files — no exceptions.
- No any types anywhere in the codebase. Use unknown and type-narrow, or define proper types.
- All shared types live in packages/types/. Module-specific types live in the module's types/ folder.
- Zod infer is the source of truth for API input/output types — never write duplicate interfaces.
- All async functions return typed Promises. No implicit void returns on async functions.

11.2 Naming Conventions

Item	Convention	Example
Files (component)	PascalCase	EmployeeProfile.tsx
Files (utility/hook)	camelCase	useEmployee.ts, formatDate.ts
Files (backend)	camelCase	employee.service.ts, auth.routes.ts
React Components	PascalCase	EmployeeProfile, BPVScoreCard
Hooks	useNoun	useEmployee, useBPVScore
Services	verbNoun	createEmployee(), calculateBPV()
Query keys	nounKeys object	employeeKeys.detail(id)
DB columns	camelCase (Prisma)	fullName, effectiveFrom, createdAt
Env variables	SCREAMING_SNAKE	JWT_ACCESS_SECRET, DATABASE_URL
API endpoints	kebab-case nouns	/time-entries, /bpv-scores
Error codes	SCREAMING_SNAKE	VALIDATION_ERROR, TOKEN_EXPIRED

11.3 Backend Rules

- Controllers must never contain business logic — they only call service functions and return responses.
- Services must never access req or res objects — they are framework-agnostic.
- All database access goes through Prisma — no raw SQL queries unless absolutely required (document reason).
- All configuration is read through ConfigService — never query config tables directly in a service.
- All errors are thrown as AppError instances with a statusCode and a code string.
- No console.log in production code. Use the logger utility.

11.4 Frontend Rules

- No data fetching in components — all queries go through TanStack Query hooks in src/hooks/.
- No direct API calls from components — always use the Axios instance from src/lib/api.ts.
- No client-side state for server data — TanStack Query is the single source of truth for server state.

- All form state managed with React Hook Form. No useState for form fields.
- All toast notifications called from mutation callbacks — not from components directly.
- No hardcoded strings for roles, statuses, or config keys — use shared constants from packages/types.

12. Local Development Setup

```
# 1. Clone and install
git clone <repo-url> hrms
cd hrms
bun install                                # installs all workspace packages

# 2. Start PostgreSQL
docker-compose up -d                      # starts PostgreSQL on port 5432

# 3. Configure environment
cp backend/.env.example backend/.env
cp frontend/.env.example frontend/.env.local
# edit both files with your values

# 4. Run database migrations
cd backend
bun prisma migrate dev                    # applies all migrations, generates Prisma
client                                    client
bun prisma db seed                        # seeds master config data

# 5. Start dev servers
cd .. # back to monorepo root
bun dev                                  # starts both frontend and backend
concurrently

# Frontend: http://localhost:3000
# Backend:  http://localhost:4000
```

13. Document Control

Field	Value
Document Title	HRMS — Technical Design Document
Version	v1.0
Status	Draft — For Review
Date	July 2026
Classification	Confidential
Companion Documents	SDD v2.0 (Solution Design), FDD v1.0 (Functional Design)
Intended Audience	Frontend Engineers, Backend Engineers, DevOps, Tech Leads

All architectural decisions documented here are subject to revision during implementation. Deviations must be documented in an Architecture Decision Record (ADR) and reviewed by the Tech Lead.